



THE UNIVERSITY of EDINBURGH
informatics

HW/SW Co-designed Processors: Challenges, Design Choices and a Simulation Infrastructure for Evaluation

**Rakesh Kumar¹, José Cano¹, Aleksandar Brankovic², Demos Pavlou³,
Kyriakos Stavrou³, Enric Gibert⁴, Alejandro Martínez⁵, Antonio González⁶**

¹University of Edinburgh, UK ²Intel ³11pets

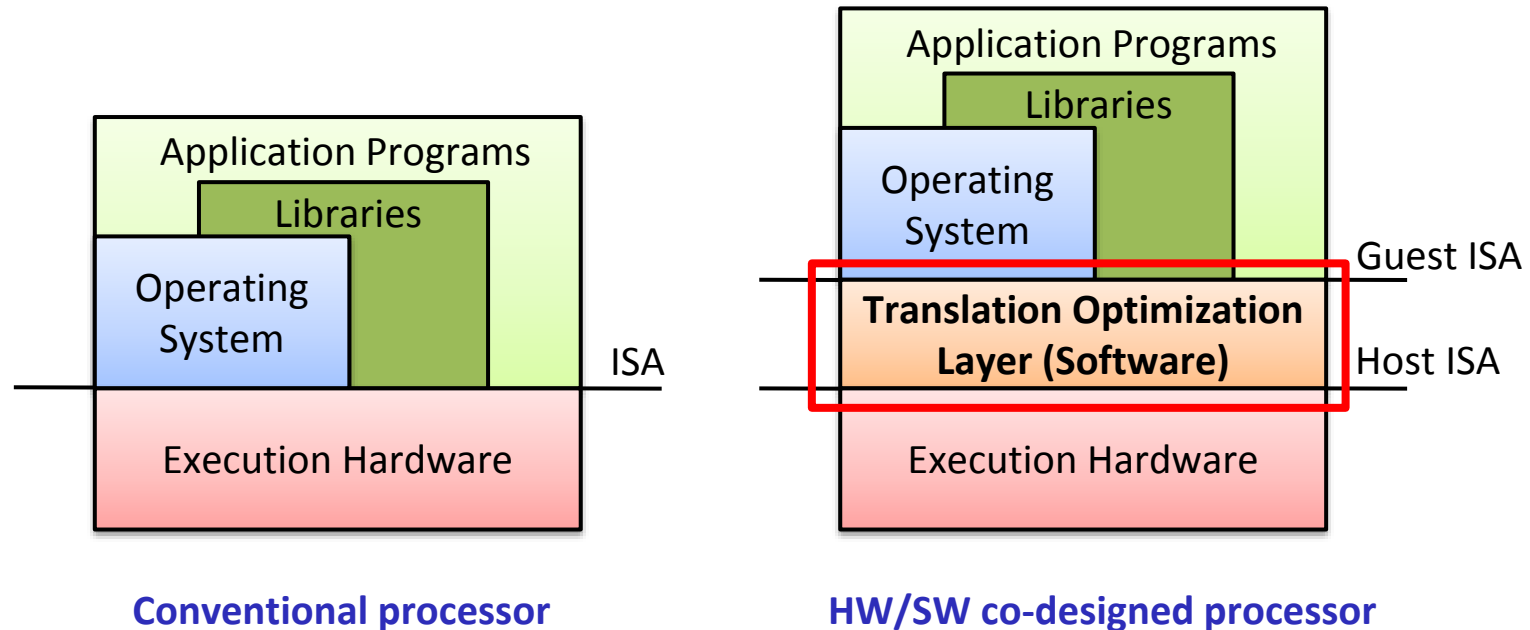
⁴Pharmacelera ⁵ARM ⁶Universitat Politècnica de Catalunya, Spain

IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)
Santa Rosa, California, USA - April 24-25, 2017

Outline

- HW/SW co-designed processors
- Building a simulation infrastructure
- DARCO
- Evaluation
- Conclusions

HW/SW co-designed processors



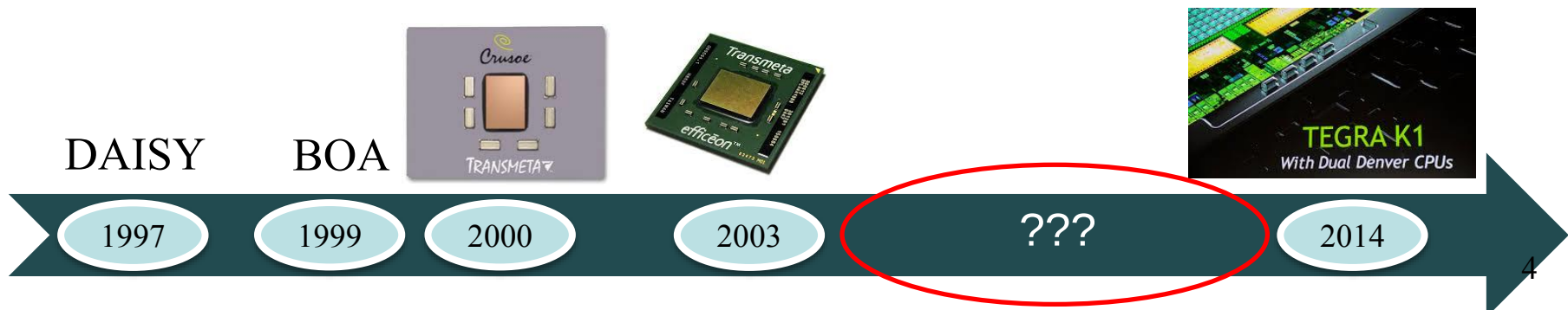
- **Simple Host ISA**
 - In-order cores; move complexity to software layer
- **Dynamic Binary Optimizations in software (TOL)**
 - Aggressive and speculative
 - Exploit application behavior at runtime

Energy Efficiency

Performance

HW/SW co-designed processors: History

- IBM **DAISY** (1997)
 - Targets **binary compatibility** from PowerPC to VLIW architectures
- IBM **BOA** (1999)
 - Targets **high frequency PowerPC** through simple hardware design
- Transmeta **Crusoe** (2000) and **Efficeon** (2003)
 - Execute x86 binaries on proprietary VLIW with **low power consumption**
 - Better energy efficiency than Intel Pentium III
- Nvidia **Denver** (2014)
 - Executes ARMv8 binaries on proprietary in-order core
 - Applying dynamic optimizations **matches Out-of-order** Intel Haswell



HW/SW co-designed processors: History

- IBM **DAISY** (1997)
 - Targets **binary compatibility** from PowerPC to VLIW architectures
- IBM **BOA** (1999)

Anything missing?

No major academic project!

Can lack of simulation infrastructure be the reason?

- Executes ARMv8 binaries on proprietary in-order core
- Applying dynamic optimizations **matches Out-of-order** Intel Haswell

DAISY

BOA



1997

1999

2000

2003

???

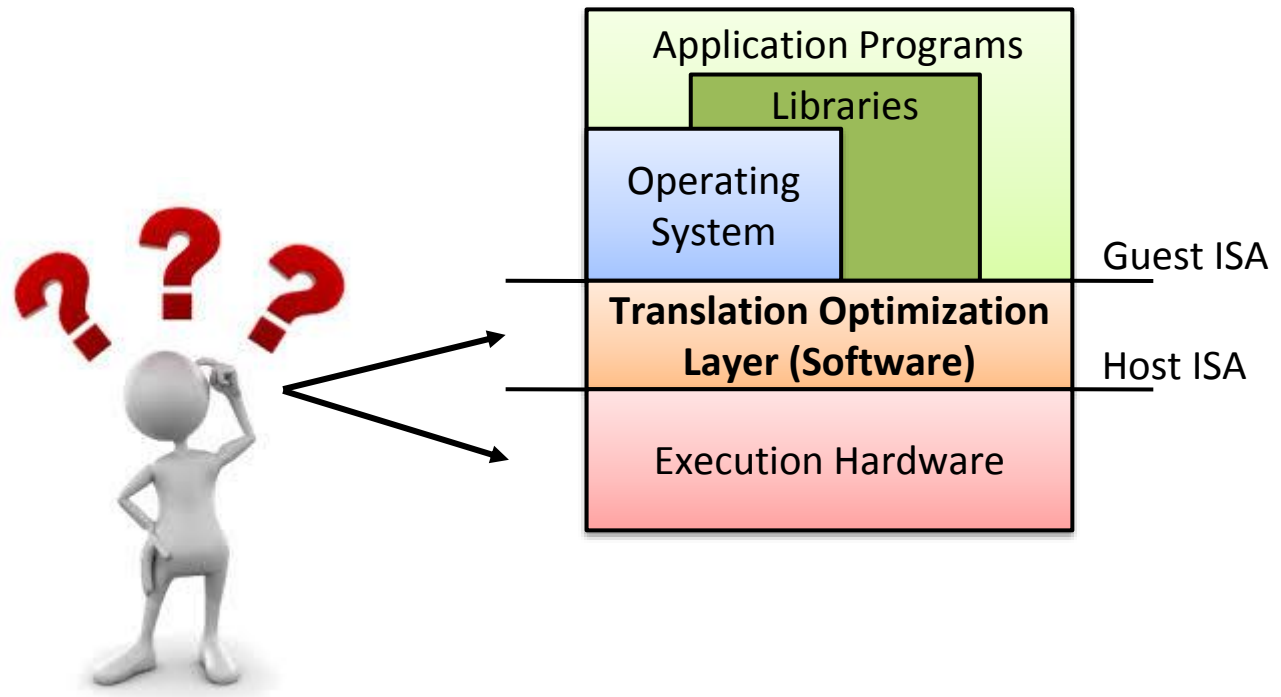
2014

Outline

- Introduction
- HW/SW co-designed processors
- **Building a simulation infrastructure**
- DARCO
- Evaluation
- Conclusions

What will a simulation infrastructure enable?

- Where to implement (HW or SW?) **microarchitectural features** like
 - Instruction decoding/reordering, memory disambiguation, register renaming, ...



What will a simulation infrastructure enable?

- Where to implement (HW or SW?) **microarchitectural features** like
 - Instruction decoding/reordering, memory disambiguation, register renaming, ...
- How to reduce “**startup delay**”
 - One of the major problems of *Transmeta* products
- When and where to **translate/optimize** the guest binaries
 - As soon as code becomes “hot”?, wait for a core to become idle?,...
- How to address **speculative** execution (memory, control)
 - Checkpointing granularity?, find susceptible speculation failure points?
- When and how to **profile** the execution
 - Overhead vs opportunity for improvement

What will a simulation infrastructure enable?

- Where to implement (HW or SW?) **microarchitectural features** like
 - Instruction decoding/reordering, memory disambiguation, register renaming, ...
- How to reduce “**startup delay**”

A simulation infrastructure can help evaluate trade-offs and design choices

- How to address **speculative** execution (memory, control)
 - Checkpointing granularity?, find susceptible speculation failure points?
- When and how to **profile** the execution
 - Overhead vs opportunity for improvement

Simulation infrastructure: Complexity

- **Compilation framework**
 - Code analysis/translation
 - Optimizations
 - Code generation
- **Runtime system**
 - Profiling and instrumentation
 - Profile-guided optimizations
- **Microarchitectural simulator**
 - Model components like pipeline, caches, ...
 - Allow sampling

Simulation infrastructure: Complexity

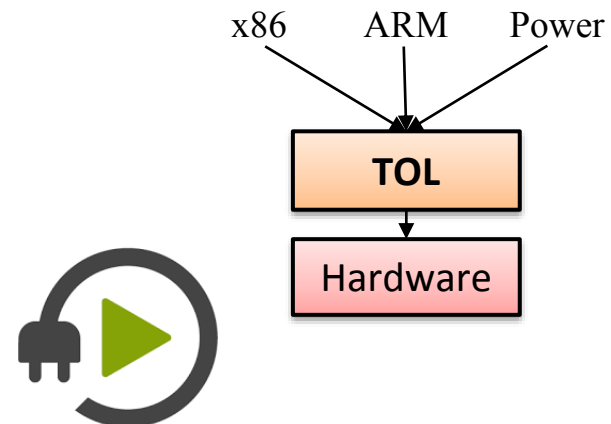
- **Compilation framework**
 - Code analysis/translation
 - Optimizations
 - Code generation

**Simulation infrastructure complexity =
Compilation framework + Runtime system +
Microarchitectural simulator**

- **Microarchitectural simulator**
 - Model components like pipeline, caches, ...
 - Allow sampling

Simulation infrastructure: Requirements

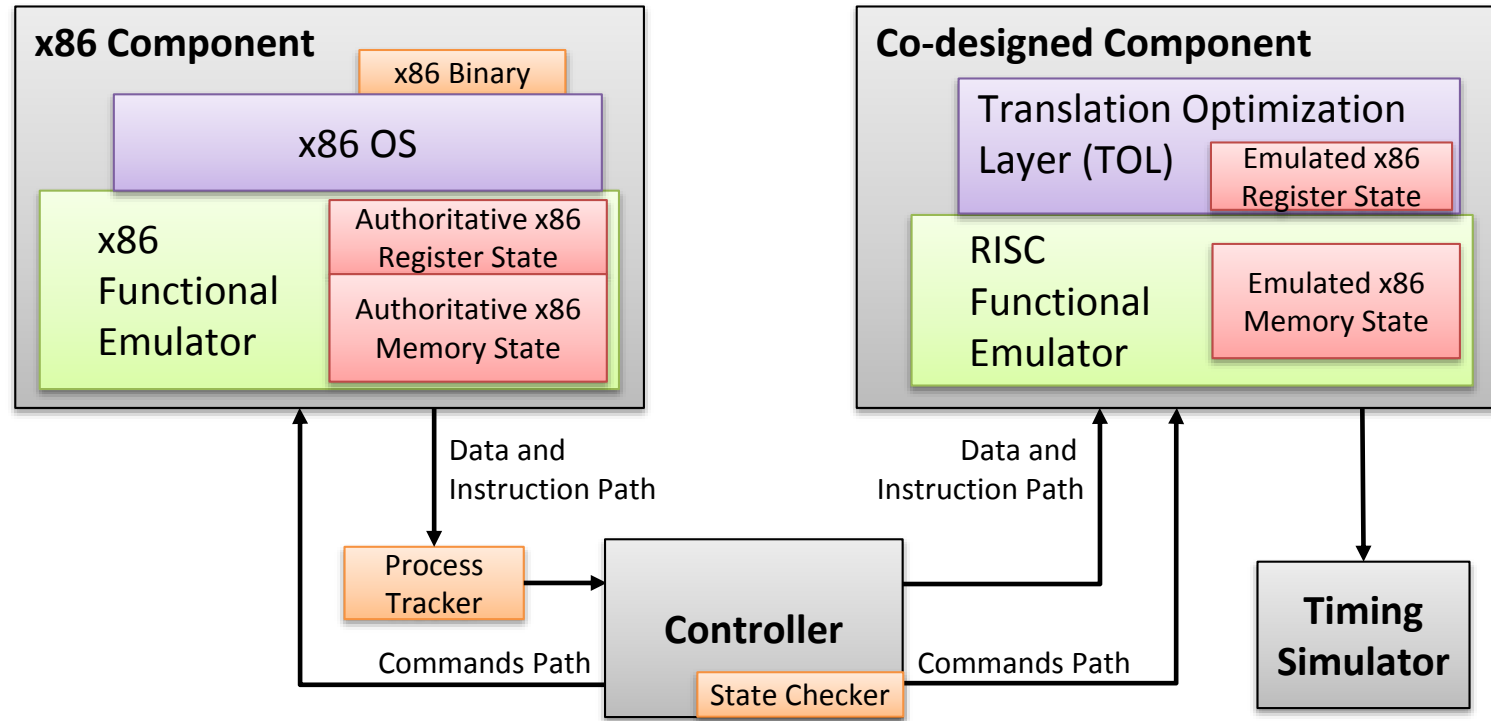
- Correctness
 - It should not change program behavior
- Minimum software layer (TOL) overhead
 - TOL execution time must be small
- Minimum emulation cost
 - Host to guest instruction ratio must be low
- Support for multiple guest ISAs (front-ends)
 - Enables wider applicability
- Plug and play support
 - Easy to include/evaluate new features
- Debugging
 - Strong debug toolchain



Outline

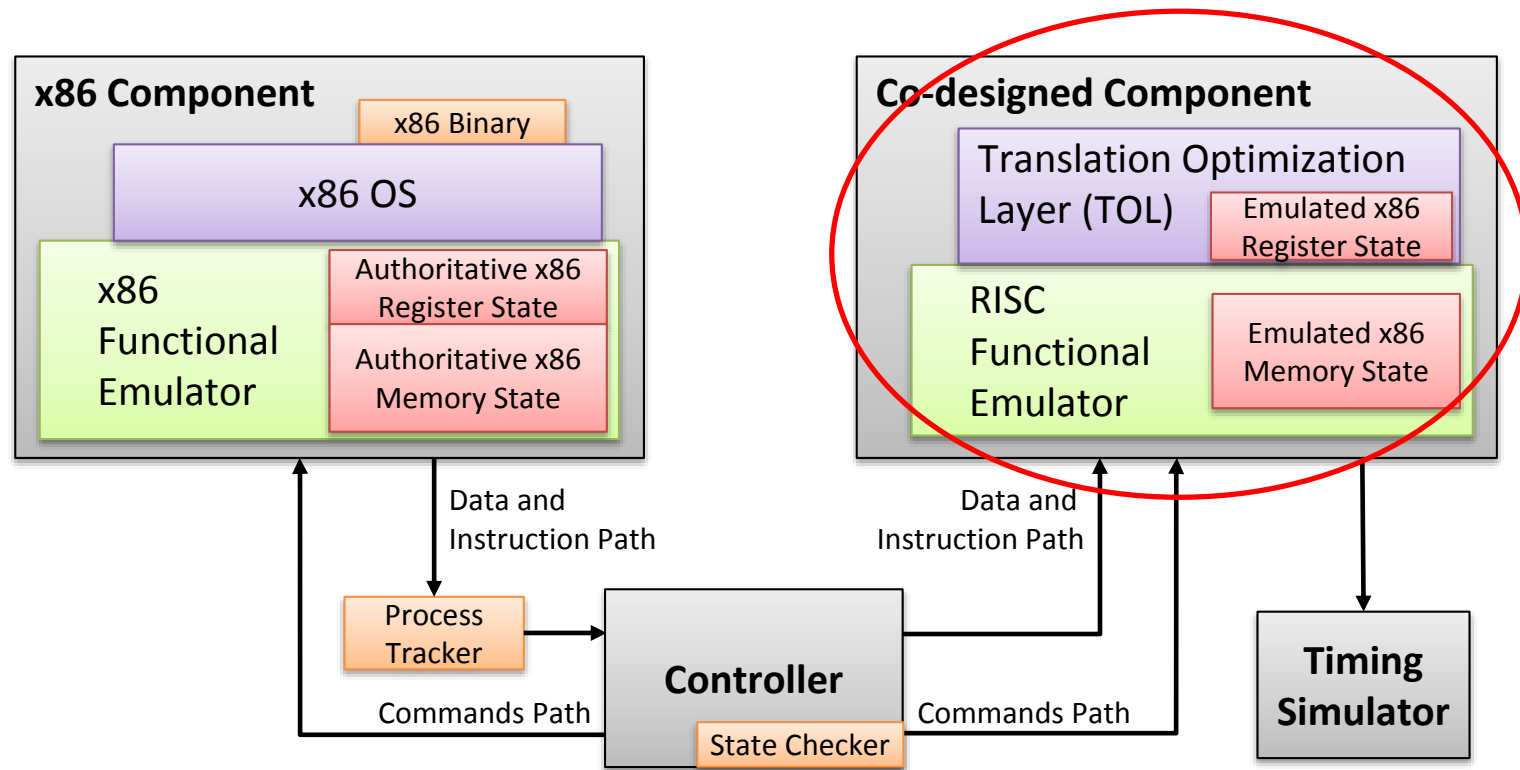
- Introduction
- HW/SW co-designed processors
- Building a simulation infrastructure
- **DARCO: Infrastructure for Research on HW/SW Co-designed Processors**
- Evaluation
- Conclusions

DARCO: The big picture



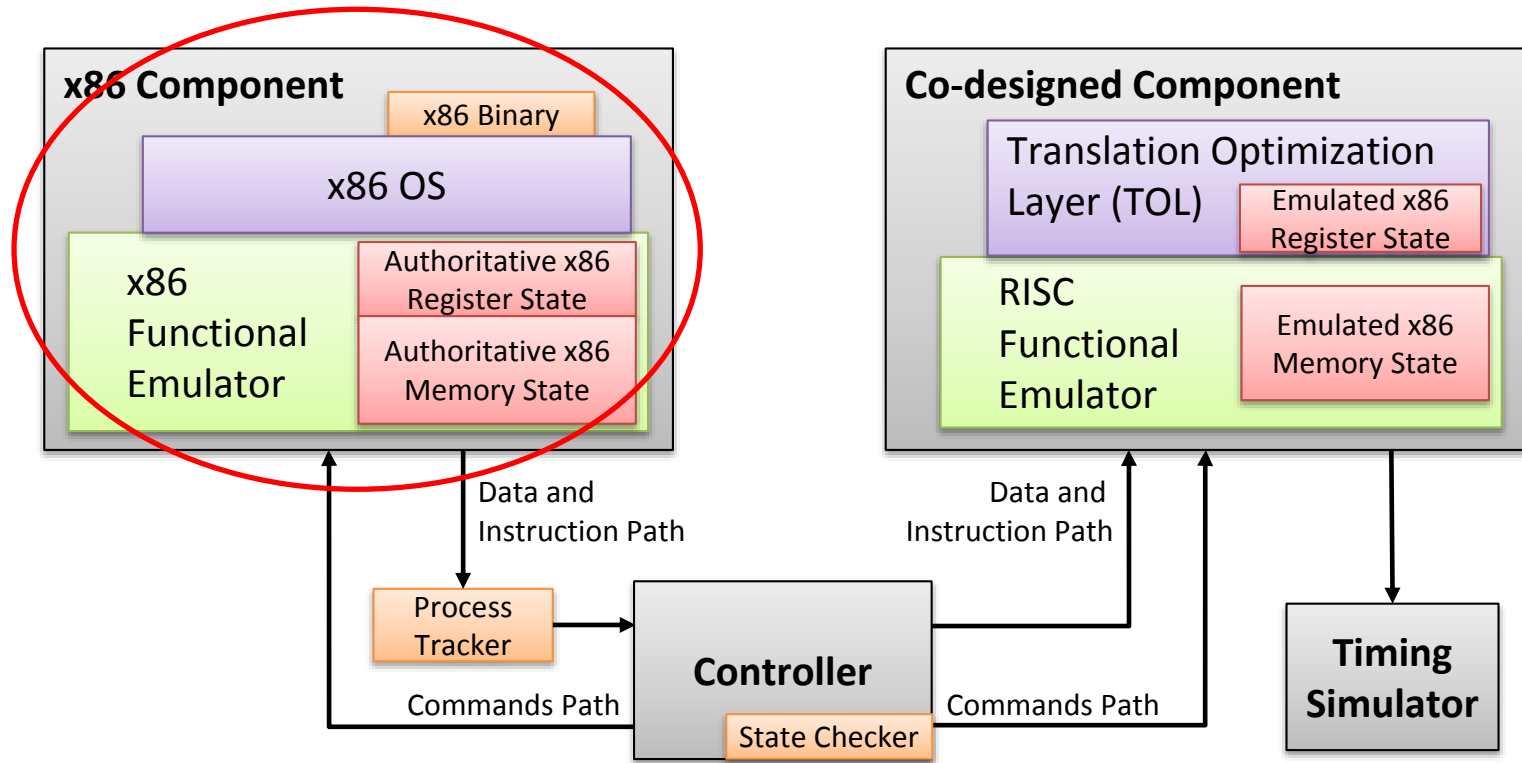
- Models a processor that **executes x86 code on a RISC host** architecture
- **Four main components:** Co-designed, x86, Timing Simulator, Controller

DARCO: Co-designed Component



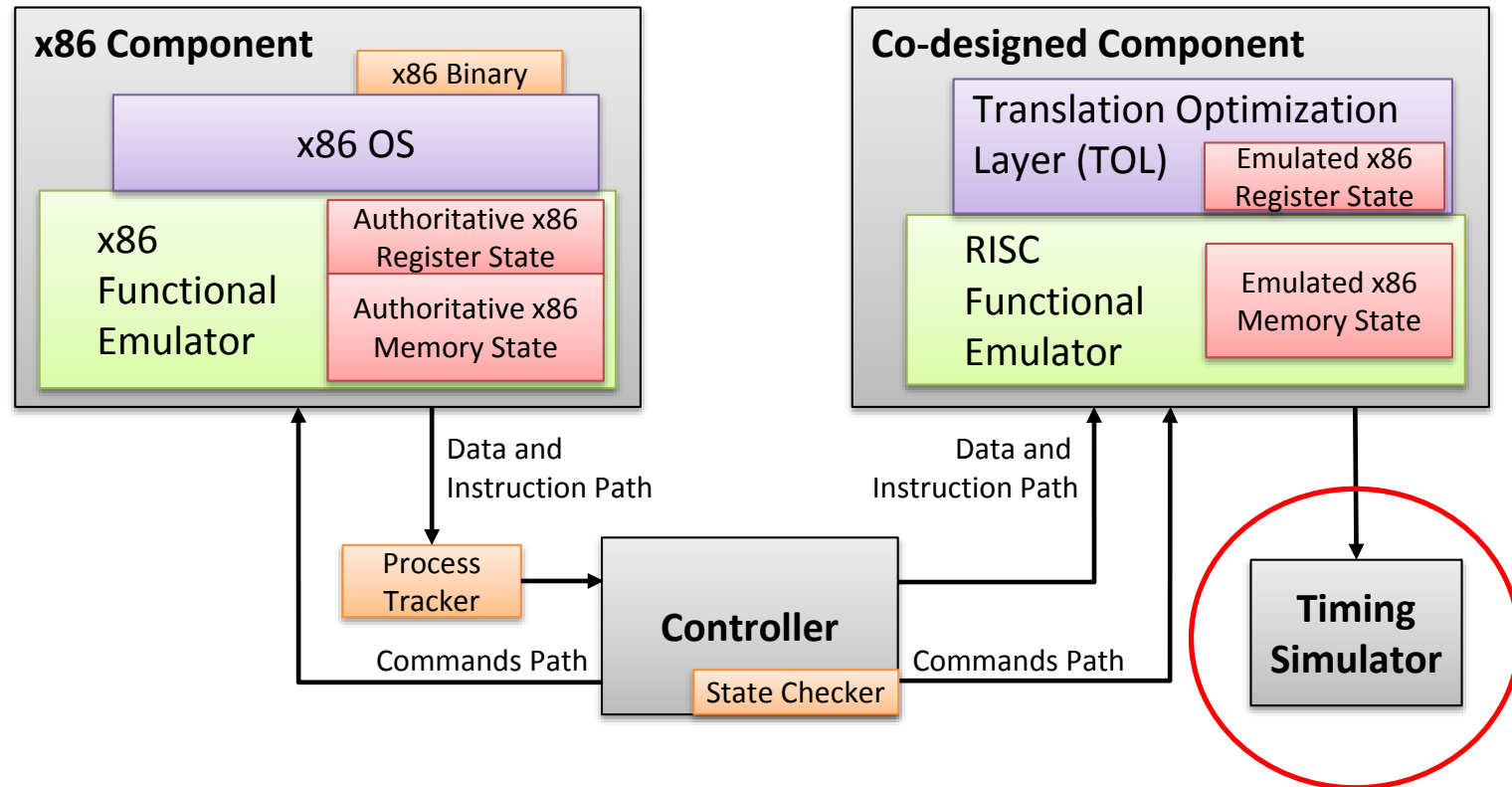
- Models the functionality of a HW/SW co-designed processor
- Composed of **TOL** and **host ISA functional emulator** (user code)
- Maintains emulated x86 architectural and memory states

DARCO: x86 Component



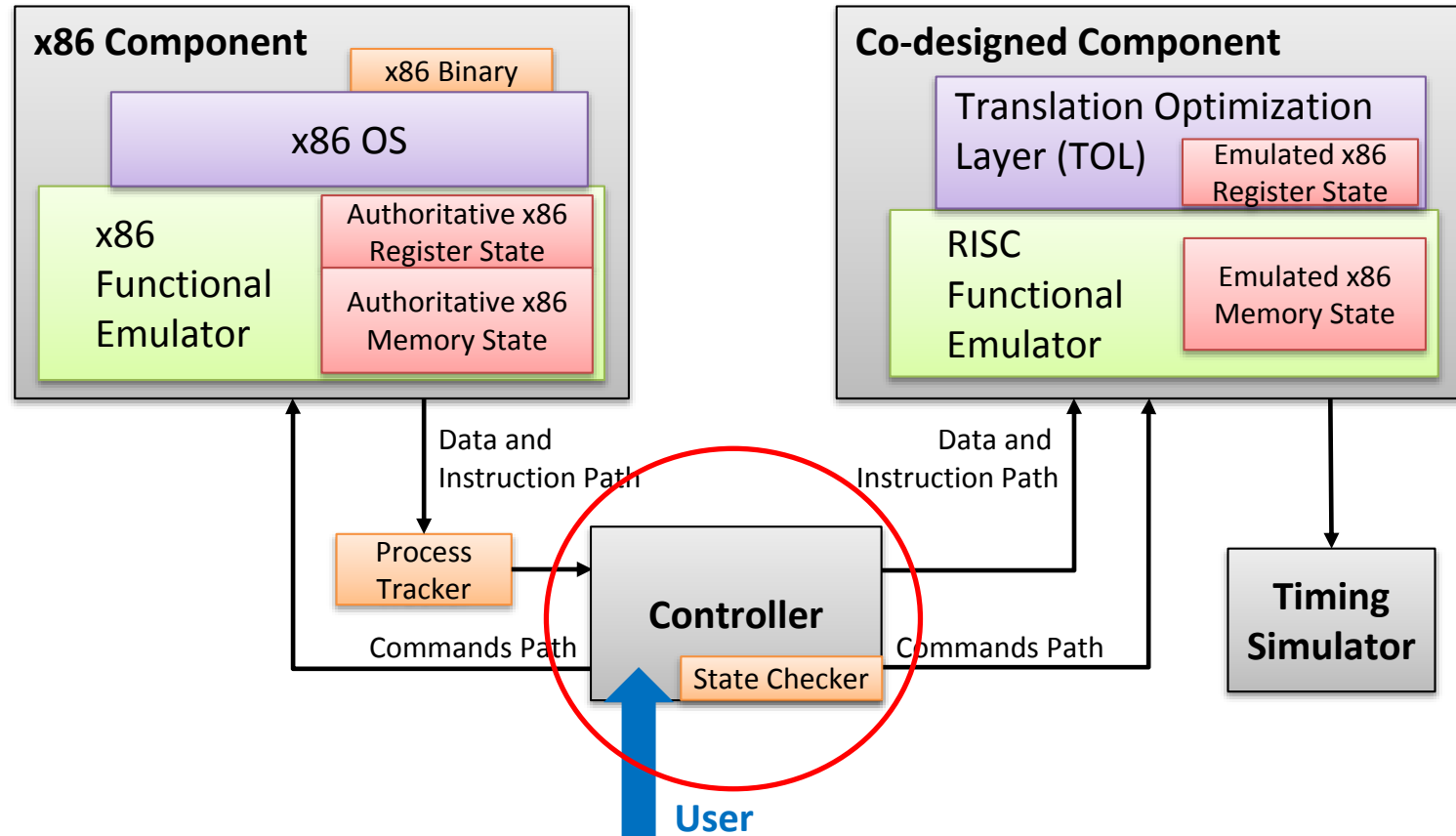
- Provides a **full-system functional emulator** for the guest x86 ISA
- Maintains authoritative x86 architectural and memory states
- Filters instruction stream and passes user code to co-designed component

DARCO: Timing Simulator



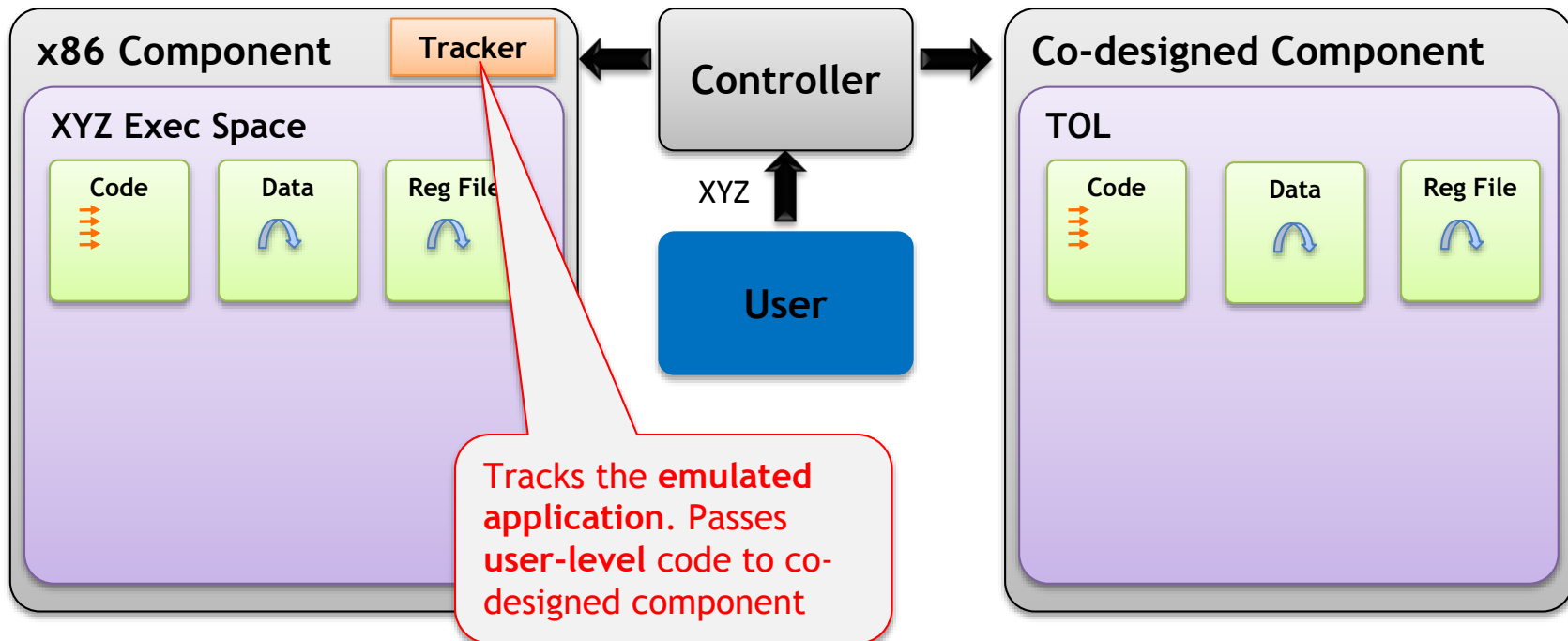
- Models a parameterized in-order core
- Can distinguish application and TOL code
- Includes power and energy modelling (McPAT)

DARCO: Controller



- Provides full control over the app execution and debugging utilities
- Compares authoritative and emulated x86 states to ensure correctness

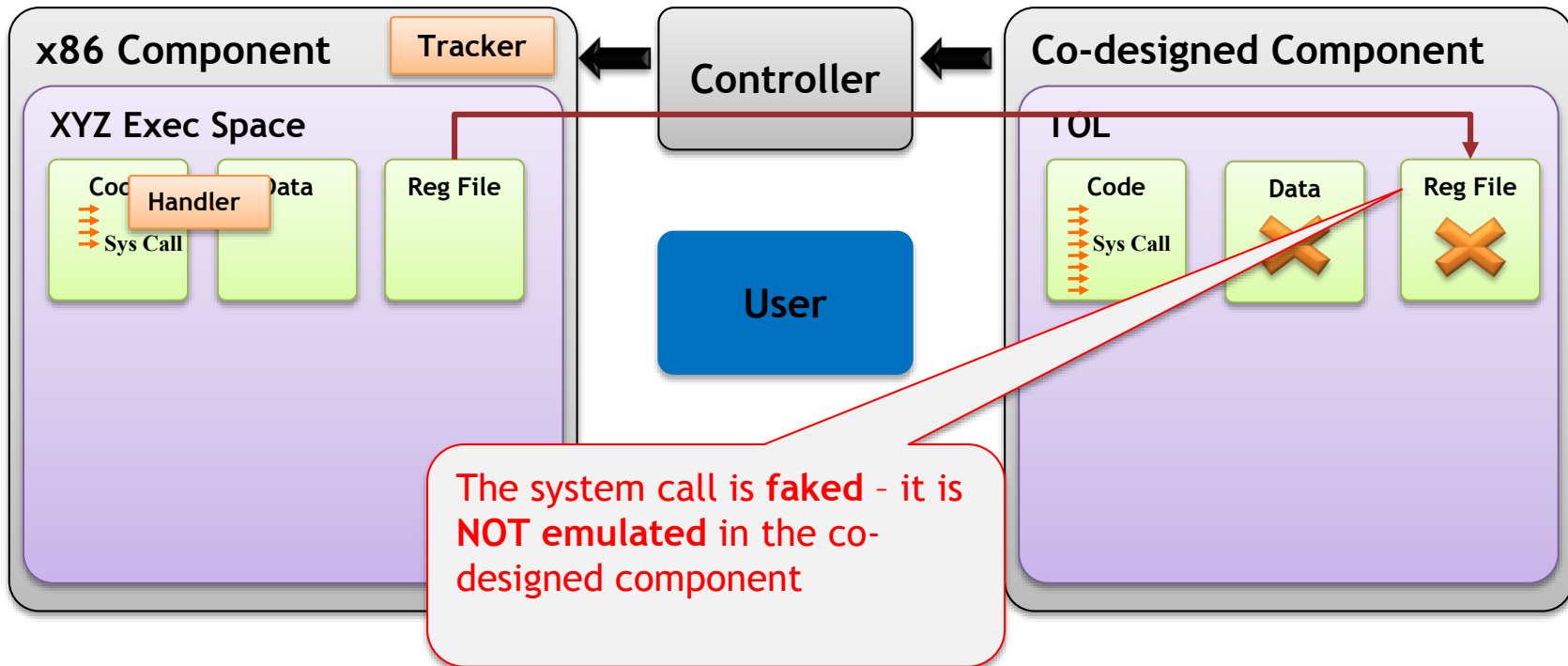
DARCO: Starting execution



Commands

- User** : Execute application XYZ
- Controller** : Send proper command to x86 component
- x86 OS** : Starts application XYZ
- Tracker** : Identifies application XYZ
- Controller** : Request 1st code page from x86 component and send to TOL with initial state
- TOL** : Load the code page and starts emulating

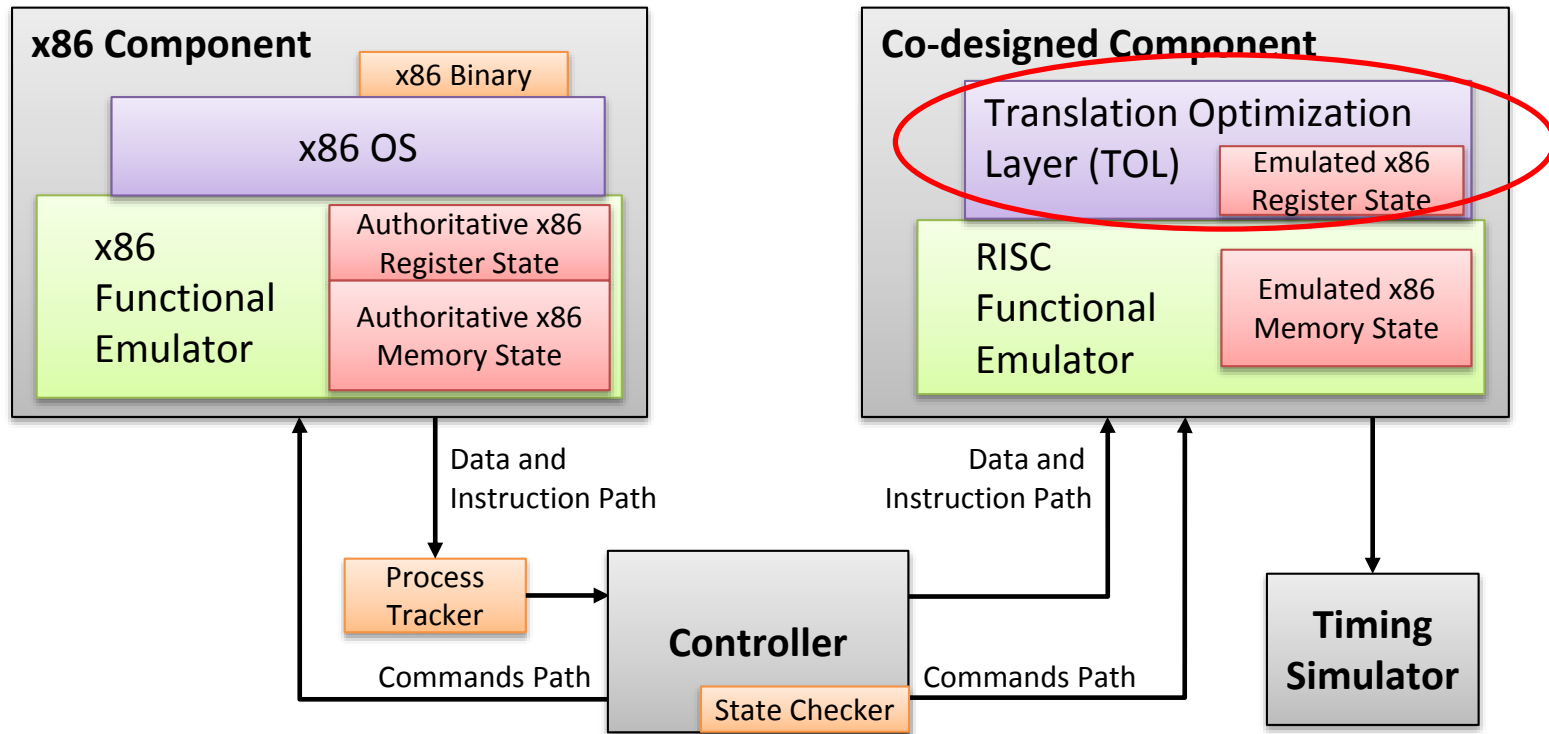
DARCO: Handling system calls



Events and Commands

- TOL** : Execution sequence reaches a system call
- TOL** : Flush state and send request to x86 component
- x86 Comp** : Reach the same execution point and make the system call
- Controller** : Send new state to TOL
- TOL** : Continue emulation

DARCO: Translation Optimization Layer (TOL)



DARCO: TOL execution modes

- **Interpretation (IM)**

- x86 instructions interpreted sequentially
- Profiling for execution frequency of basic blocks

Cold Code

- **Basic block translation (BBM)**

- x86 basic blocks translated to intermediate representation
- Lightweight optimizations (dead code, constants)
- Profiling for branch directions and execution frequency of basic blocks

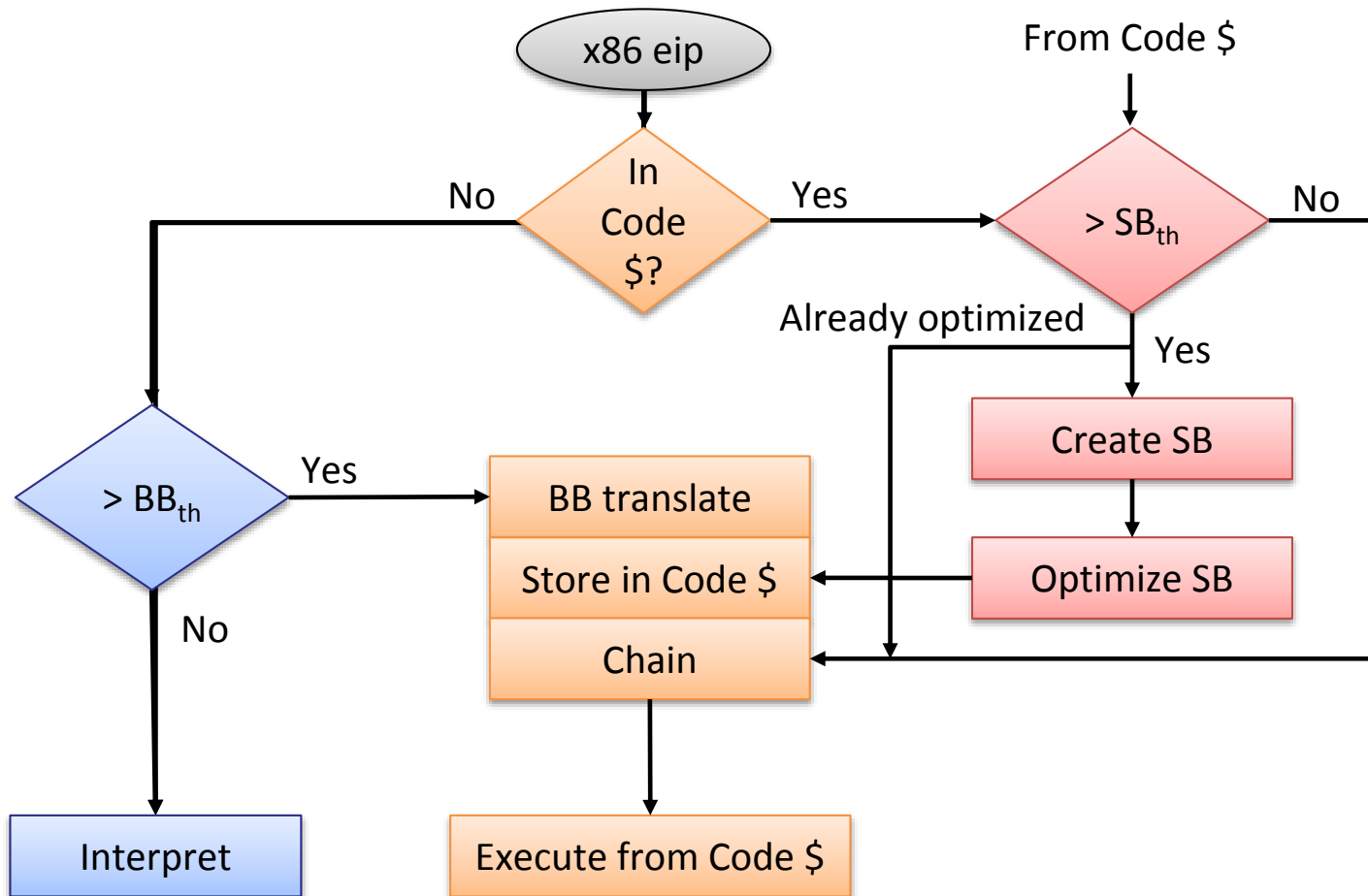
Warm Code

- **Superblock optimization (SBM)**

- Bigger optimization regions across multiple x86 basic blocks
- Aggressive and speculative optimizations
- No profiling (reduces overhead)

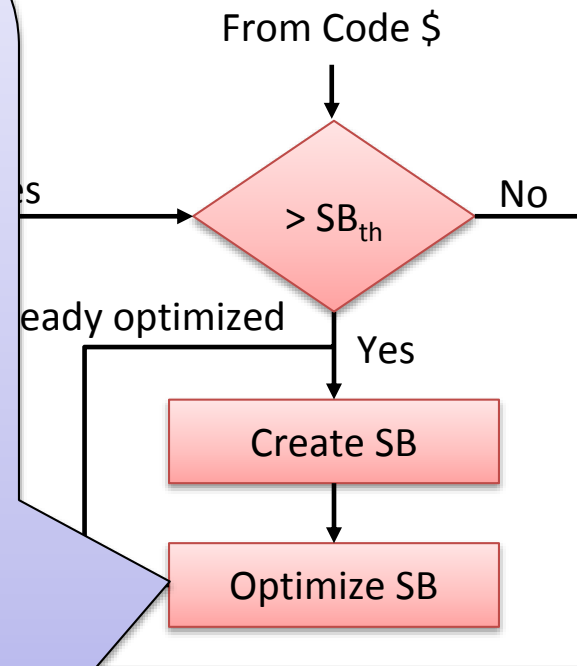
Hot Code

DARCO: TOL execution flow

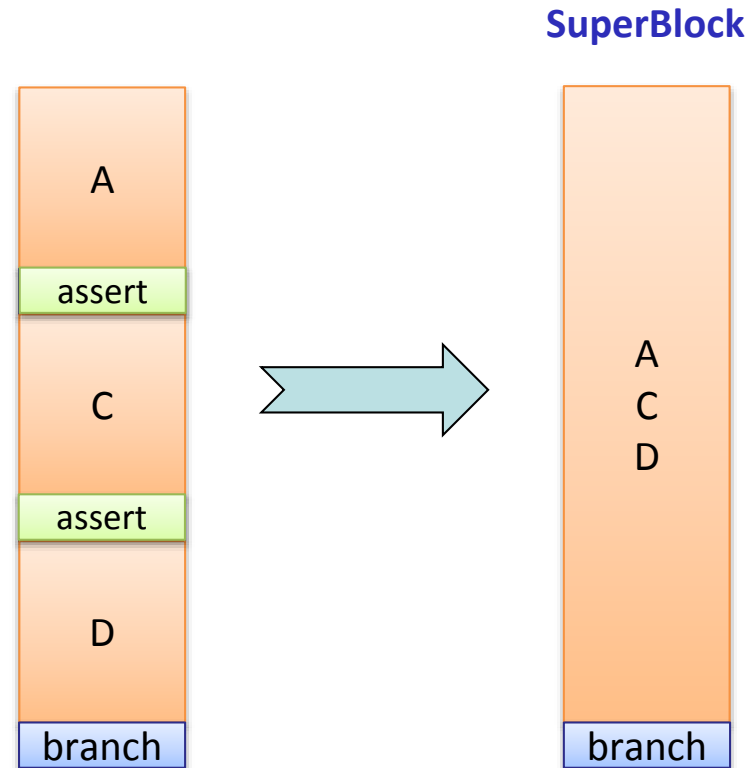
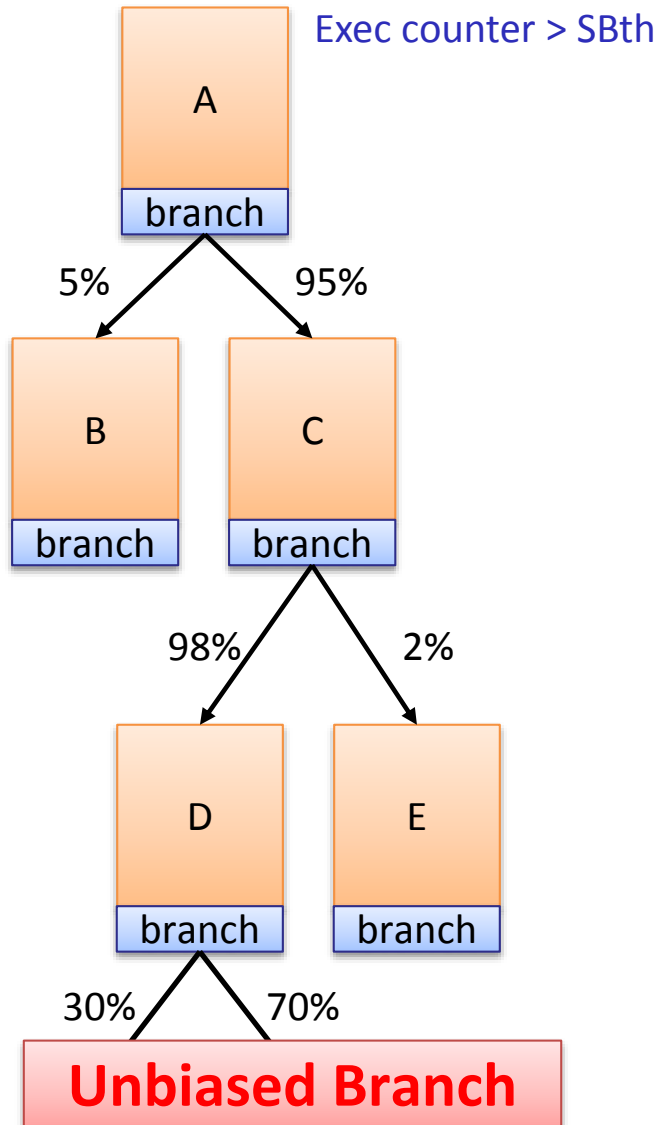


DARCO: TOL execution flow

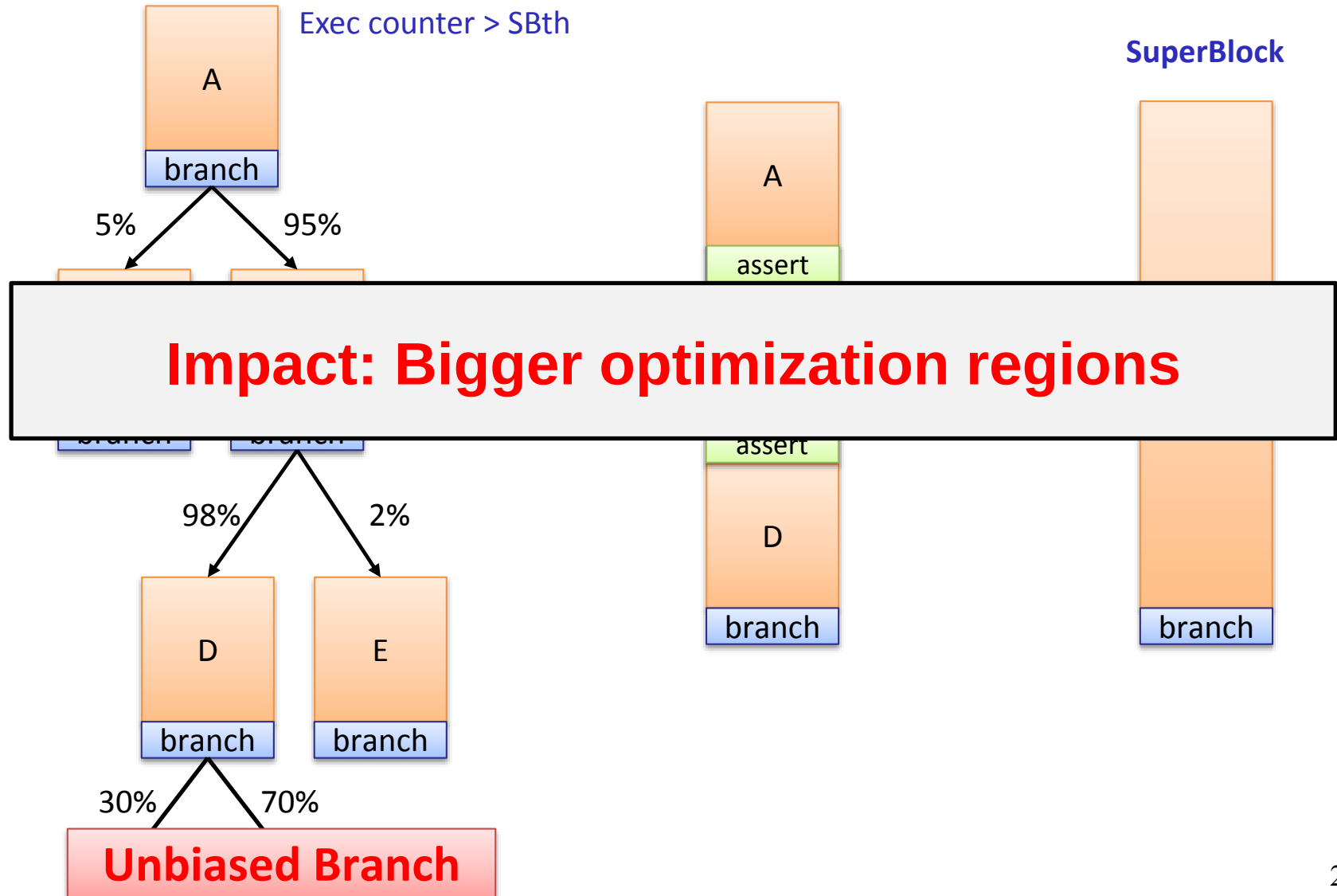
- x86 to Intermediate Representation (IR)
 - Control Speculation
 - Loop Unrolling
- Static Single Assignment (SSA)
- Forward Pass
 - Constant Folding
 - Constant/Copy Propagation
 - Common Subexpression Elimination
- Backward Pass
 - Dead Code Elimination
- Data Dependence Graph (DDG)
 - Memory Alias Analysis
 - Redundant Load Removal
 - Store Forwarding
- Instruction Scheduling
 - Data Speculation
- Register Allocation
- Code Generation (opt. IR to Host ISA)



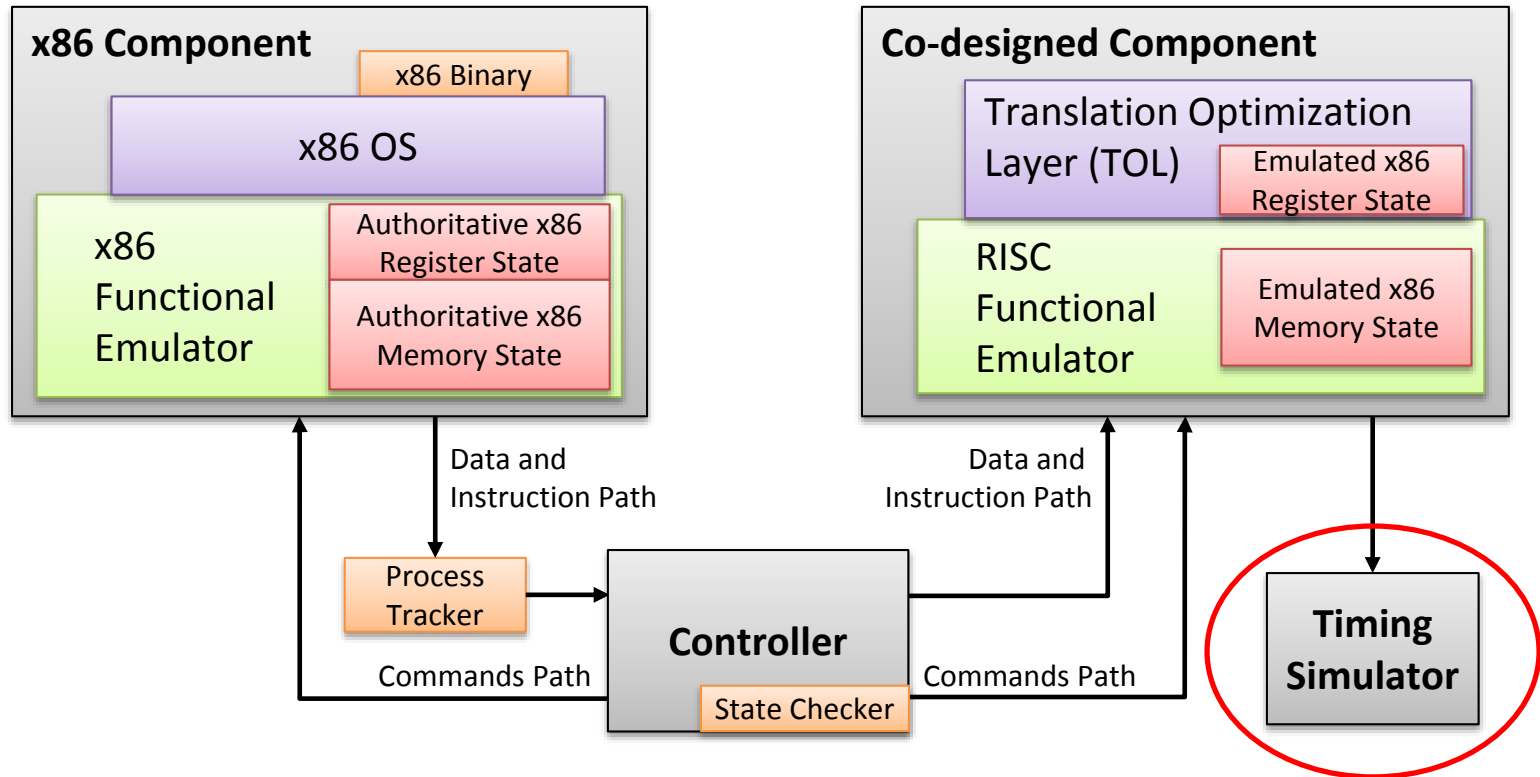
DARCO: Building a superblock



DARCO: Building a superblock

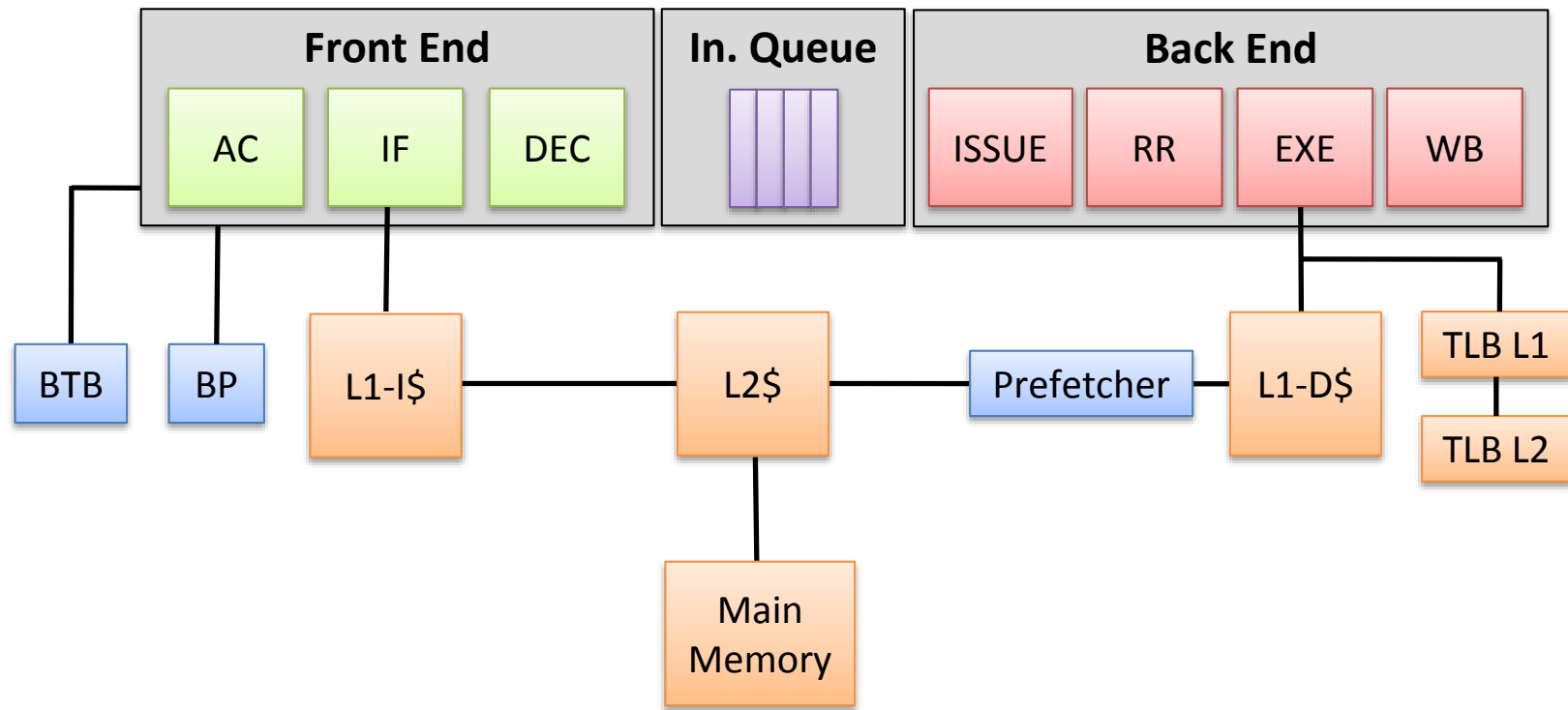


DARCO: Timing Simulator



DARCO: Timing Simulator

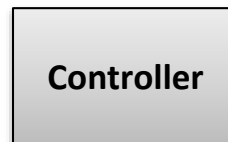
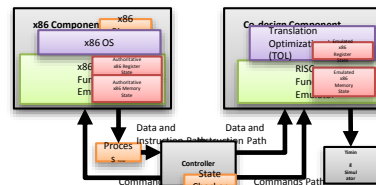
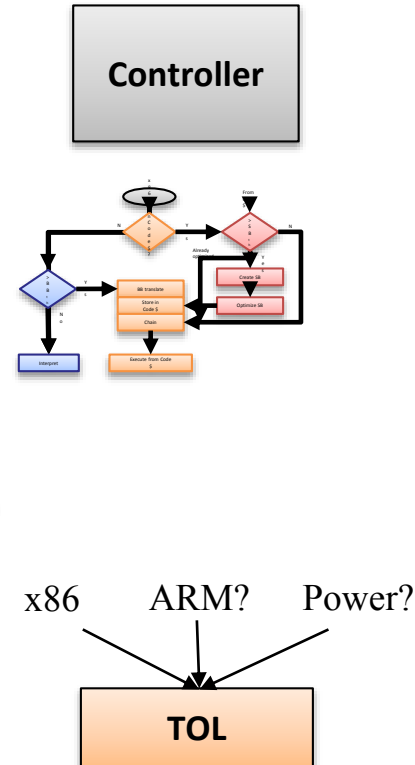
- Models a configurable **RISC** superscalar in-order processor
- Pipeline decoupled into: Front-End, Instruction Queue, Back-End



DARCO: Meeting the requirements

- Correctness
 - Architectural/memory states compared periodically
- Minimum TOL overhead
 - 3-stages for translation/optimization; Chaining
- Minimum emulation cost
 - Aggressive/speculative optimizations
- Support for multiple guest ISAs (front-ends)
 - Incorporating additional front-ends is straightforward
- Plug and play support
 - Modular design
- Debugging
 - Powerful debug toolchain

SBM Optim.



Outline

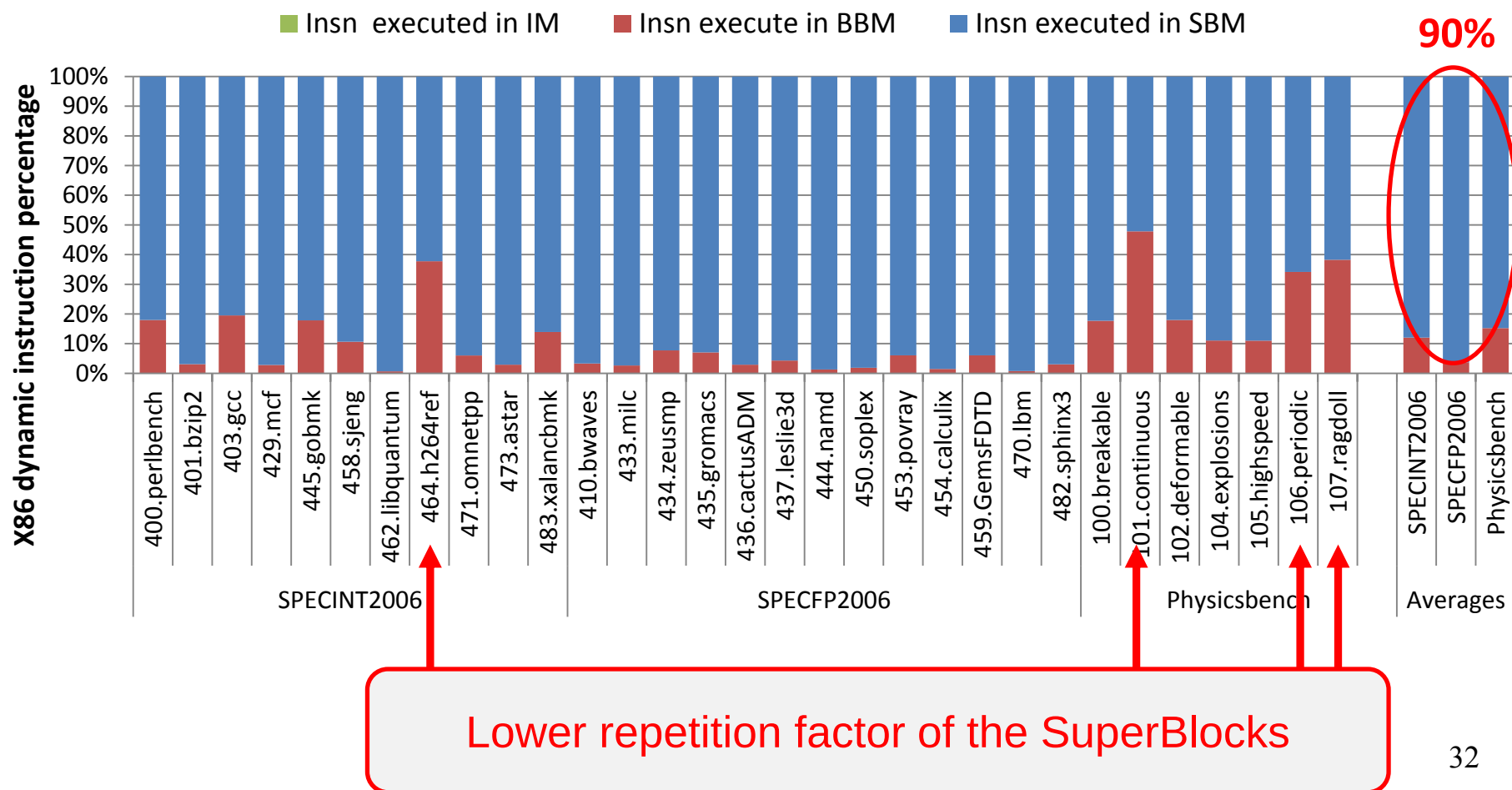
- Introduction
- HW/SW co-designed processors
- Building a simulation infrastructure
- DARCO
- **Evaluation**
- Conclusions

Evaluation methodology

- **Benchmarks** analyzed
 - SPEC CPU2006: Integer, Floating point
 - Physicsbench
- We simulate 4B (billion) x86 instructions
 - Some benchmarks have fewer instructions and run to completion
- DARCO speed (x86 instructions)
 - 3.4 MIPS and ~0.4 MIPS enabling the Timing Simulator
 - Intel Xeon E5-2630L (2.40 GHz) and L5630 Dual (2.13 GHz) with 24-128 GB RAM
- Highlights
 - x86 dynamic code distribution
 - Execution time distribution
 - Emulation cost

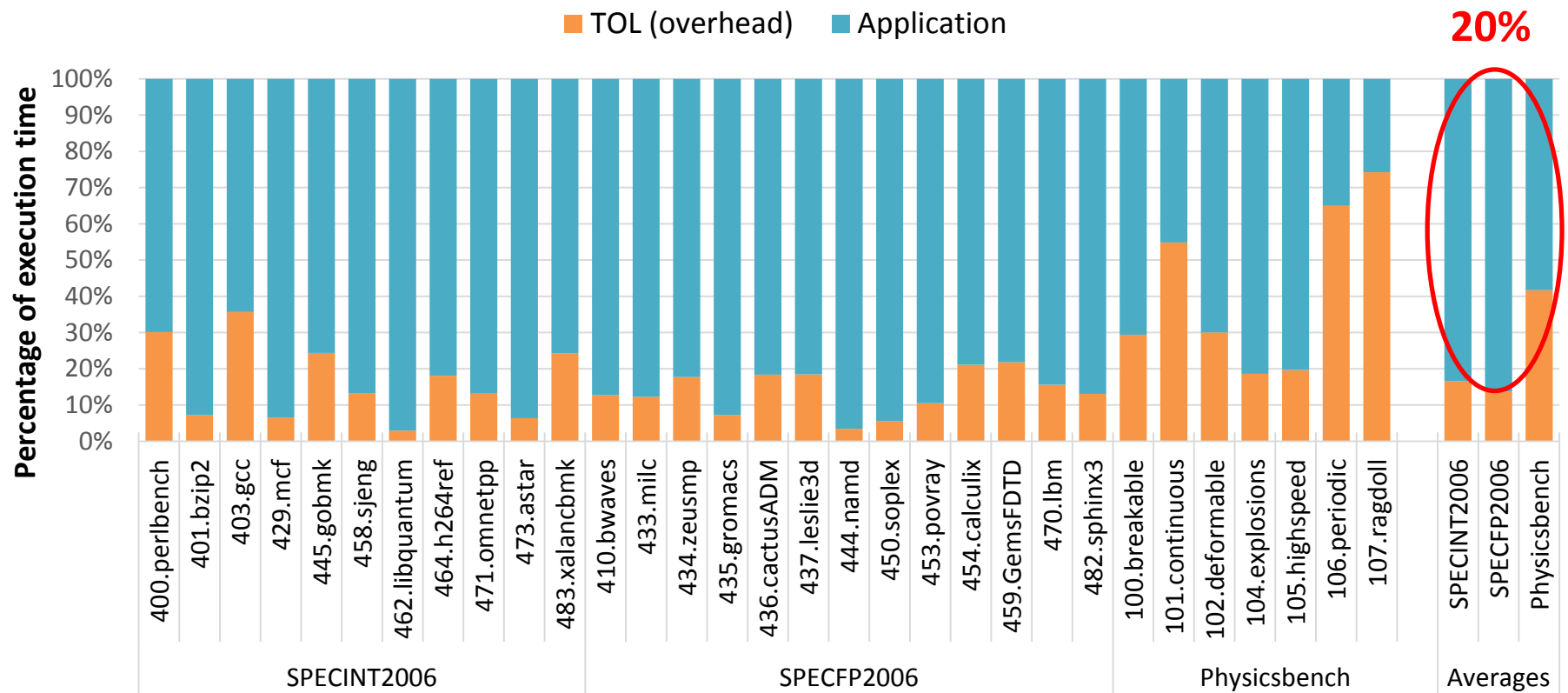
Evaluation: x86 dynamic code distribution

- ~90% of the dynamic instruction stream comes from SBM (optimized code)
- Represents ~14% of the static code (not shown)



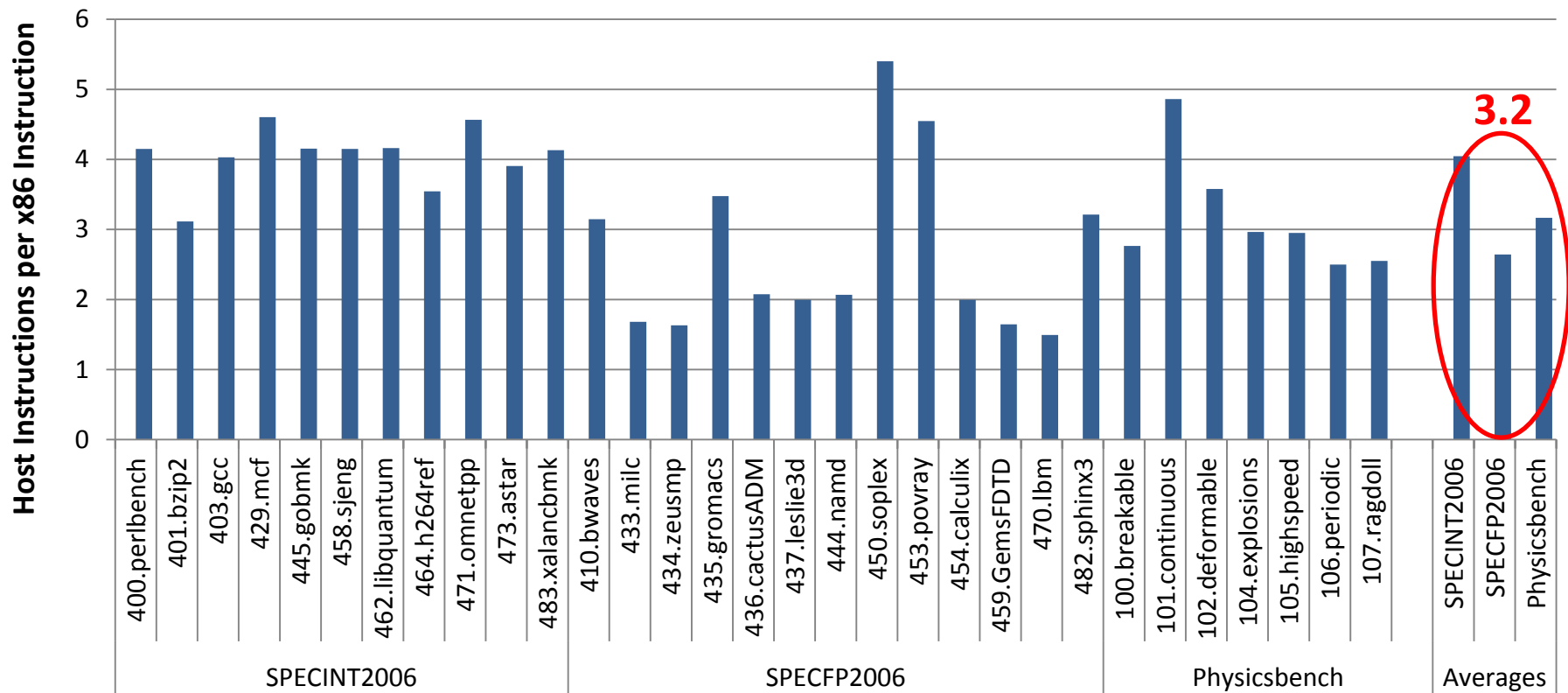
Evaluation: Execution time distribution

- **Application:** cycles executing RISC instructions that emulate the x86 code
- **TOL (overhead):** cycles translating/optimizing the guest code, etc



Evaluation: Emulation cost

- Number of host instructions generated per x86 instruction ~3
- Reasonable for translating from CISC to RISC



Conclusions

- **HW/SW co-designed processors**
 - Huge potential to improve energy efficiency and performance
 - Several industrial projects
 - No major project in academia (lack of simulation infrastructures)
- **Challenges**
 - To become mainstream (e.g. startup delay)
 - In building a simulation infrastructure (e.g. software layer overhead)
- **DARCO**
 - May enable academic research in HW/SW co-designed domain
 - Provides a modular simulation infrastructure
 - Easy to add new components/optimizations



THE UNIVERSITY of EDINBURGH
informatics

HW/SW Co-designed Processors: Challenges, Design Choices and a Simulation Infrastructure for Evaluation

**Rakesh Kumar¹, José Cano¹, Aleksandar Brankovic², Demos Pavlou³,
Kyriakos Stavrou³, Enric Gibert⁴, Alejandro Martínez⁵, Antonio González⁶**

¹University of Edinburgh, UK ²Intel ³11pets

⁴Pharmacelera ⁵ARM ⁶Universitat Politècnica de Catalunya, Spain

THANK YOU !!!